Software Performance Engineering



Recitation 1.1

Atharva Kirkole

Thursday, Sep 3, 2026

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Course Logistics



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Attendance!



Attendance for recitations is **mandatory**!

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Some due dates

The passed due dates:

- HW0 (bio sign up and course info quizlet): Tuesday, Sep 1

Upcoming due dates:

- HW1 checkoff: TODAY!
- HW1: Wednesday, Sep 9 (Released at on 6th)
- Weekly report (week 1): Wednesday, Sep 9 (Released on 4th)

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Office Hour

- No fixed office hour
- reach out during recitation & on piazza
- If you need some in-person discussion time, schedule an office hour with me!
- My email: kirkolea@msu.edu

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Semester overview

- 6 homeworks
  - follow the handout instructions
  - submit your write-up
    - Gradescope and GitHub homework repo
  - some part will be checked off in class
- 3 projects
  - project 1: invididual, project 2: assigned team
  - beta submission & beta write-up
  - final submission & final write-up

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Weekly status report

- A short paragraph of your weekly summary
- Open on Fridays, close on Mondays, submit on Gradescope
- What you can talk about:
  - Your impression of the past week's lectures and recitations
  - Your feelings regarding your project partner or the course in general.
  - Feedback on interactions you had with the course staff.
  - Reflect on how effectively you feel you are working on class assignments and how much time you're devoting.
  - Identify the aspects of the class you found most engaging or frustrating.
  - Offer constructive criticism about how the class is being taught.
  - Report on any outside events that may have impacted your work on the course

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Example 1

I had a good time transitioning into the course my first week. I found the lectures interesting but also fast-paced at times. I feel like my Project 1 teammates, and I bonded well, were able to set up work ethics as well as goals for the project and come up with a compelling team contract. I'm looking forward to working on Bit Hacks for Project 1. I appreciate the TA's and Instructor's support during the first week in ensuring we have all our environments set up correctly. I think I'm doing well on HW1, and I find that I'm able to follow along with most of the concepts discussed in class. Something I might try doing this week onwards is to revisit the slides the same day as the lecture (if they're posted by then) or at the earliest to ensure I can recollect and retain as much information as possible. I am a little worried about missing out on deadlines in this course, however, I found that forming group chats with my classmates to send in regular reminders could be a good way to stay on track.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Example 2

The first week-ish of classes were good but some of the format was a little bit confusing. So far we've had a recitation period either at the end of Wednesday or the entirety of Wednesday so I am not quite sure what the regular schedule is supposed to look like. Additionally, there weren't any directions for how to prepare for recitations so I am not sure if we are supposed to read the material before the recitation or just learn along with the activities and presentations. Today, it seemed to be around 50/50 for who read the material before the class and who didn't. In terms of extracurricular workload I think the class is manageable and instructions are clear. Side note, the first lecture of the class was very engaging and well done. It made me excited for the course so thank you.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

The late-day policy for assignments

- 3 late days total for each semester
- atomic, no subdivision (no 0.5 day)
- homework assignment and project write-up only, not for project beta and final assignment (the coding part)
- inform the TA **before the deadline**
  - make a private post on piazza with #late-days tag
- if exceeding the 3 late days: require a note from Student Support Service to accept your late work

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Summarize homework 1

- set up and test out the virtual environment
- basic C programming practice
- useful tools:
  - debugging tool: GDB, DEBUG=1, tbassert
  - memory checker: address sanitizer, valgrind
  - code coverage: llvm-cov

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Let's start project 1!



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Project 1: Bit Hacks



(a) (b)

Figure 1: (a) The original 192-by-192 monochrome image. **(b)** The image after rotation.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

```c
void rotate_bit_matrix(uint8_t *img, const bits_t N) {
  // Get the number of bytes per row in `img`
  const uint32_t row_size = bits_to_bytes(N);

  uint32_t w, h, quadrant;
  for (h = 0; h < N / 2; h++) {
    for (w = 0; w < N / 2; w++) {
      uint32_t i = w, j = h;
      uint8_t tmp_bit = get_bit(img, row_size, i, j);

      // Move a bit from one quadrant to the next and do this
      // for all 4 quadrants of the `img`
      for (quadrant = 0; quadrant < 4; quadrant++) {
        uint32_t next_i = N - j - 1, next_j = i;
        uint8_t save_bit = tmp_bit;

        tmp_bit = get_bit(img, row_size, next_i, next_j);
        set_bit(img, row_size, next_i, next_j, save_bit);

        // Update the `i` and `j` with the next quadrant's values and
        // the `next_i` and `next_j` will get the new destination values
        i = next_i;
        j = next_j;
      }
    }
  }

  return;
}
```

© 2008–2024 by the MIT 6.172 and

Basic idea: “follow the cycles”

$$\begin{matrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{matrix}$$

(a) before applying cyclic permutation

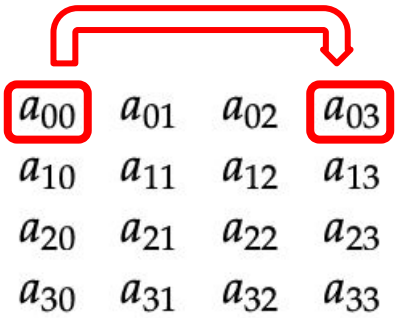
$$\begin{matrix} a_{30} & a_{01} & a_{02} & a_{00} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{33} & a_{31} & a_{32} & a_{03} \end{matrix}$$

(b) after applying cyclic permutation

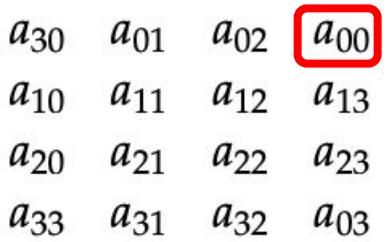
By applying the same rotation to each group of 4 bits, the program ends up rotating the entire matrix clock-wise.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Basic idea: “follow the cycles”



(a) before applying cyclic permutation

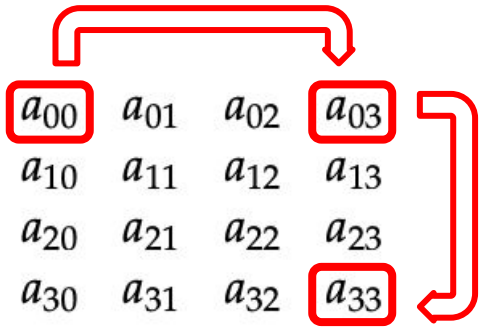


(b) after applying cyclic permutation

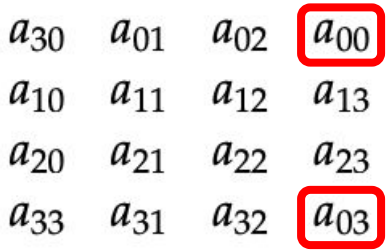
By applying the same rotation to each group of 4 bits, the program ends up rotating the entire matrix clock-wise.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Basic idea: “follow the cycles”



(a) before applying cyclic permutation

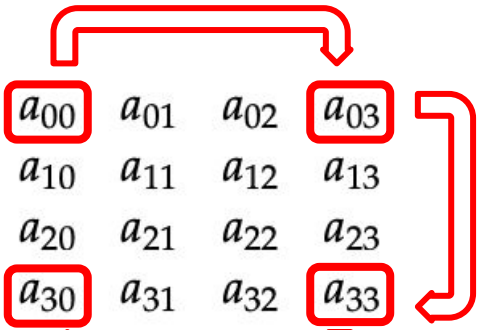


(b) after applying cyclic permutation

By applying the same rotation to each group of 4 bits, the program ends up rotating the entire matrix clock-wise.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Basic idea: “follow the cycles”



(a) before applying cyclic permutation

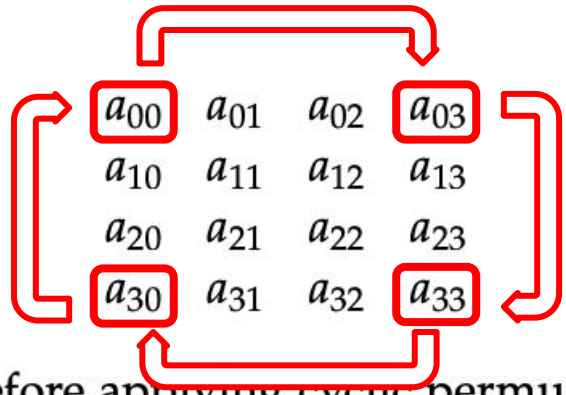
a_{30} a_{01} a_{02} a_{00}
a_{10} a_{11} a_{12} a_{13}
a_{20} a_{21} a_{22} a_{23}
a_{33} a_{31} a_{32} a_{03}

(b) after applying cyclic permutation

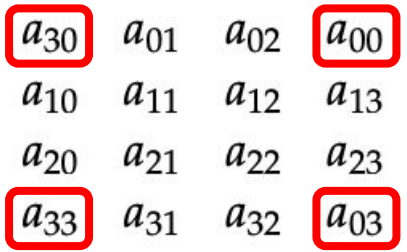
By applying the same rotation to each group of 4 bits, the program ends up rotating the entire matrix clock-wise.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Basic idea: “follow the cycles”



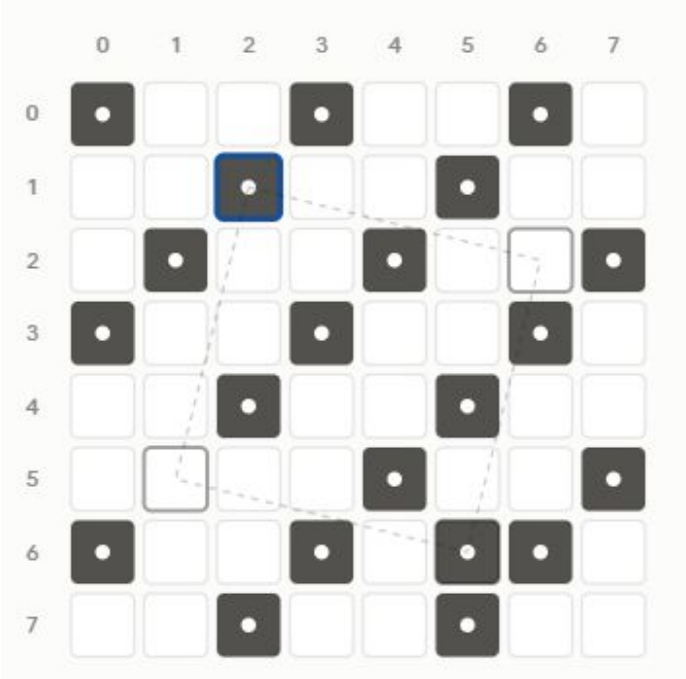
(a) before applying cyclic permutation



(b) after applying cyclic permutation

By applying the same rotation to each group of 4 bits, the program ends up rotating the entire matrix clock-wise.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Performance?

```
--- Execution log:

FYI: the max tier you can be graded on is 57.
Setting up test up to tier 25: Malloc 72192x72192 matrix...
Linear search from tier 0 to 8...
FAIL (timeout): Tier 0 :        Rotated 26624x26624     matrix in 1571 ms but the cutoff is 1000 ms
Blowing through this failure. Remaining blowthroughs: 1
PASS (ayy!):    Tier 1 :        Rotated 27712x27712     matrix in 850 ms
PASS (skrrt!):  Tier 2 :        Rotated 28864x28864     matrix in 948 ms
FAIL (timeout): Tier 3 :        Rotated 30080x30080     matrix in 1035 ms but the cutoff is 1000 ms
Blowing through this failure. Remaining blowthroughs: 0
FAIL (timeout): Tier 4 :        Rotated 31296x31296     matrix in 1167 ms but the cutoff is 1000 ms
Result: reached tier 2
```

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Some analogy



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

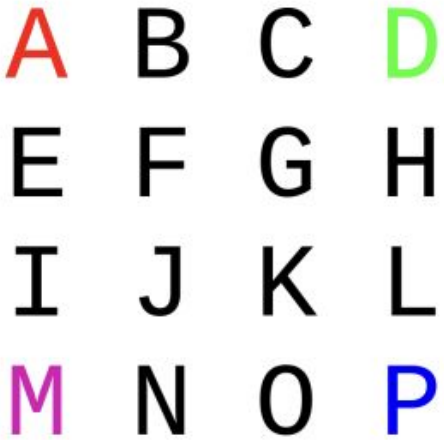
Some analogy



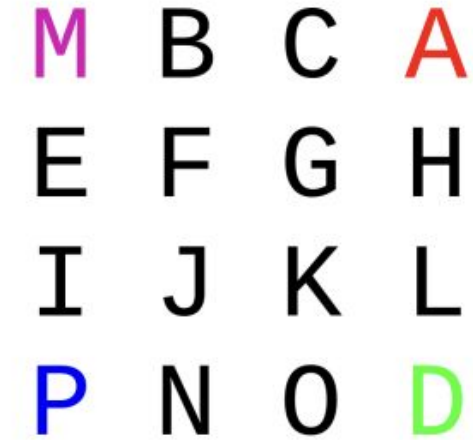


© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

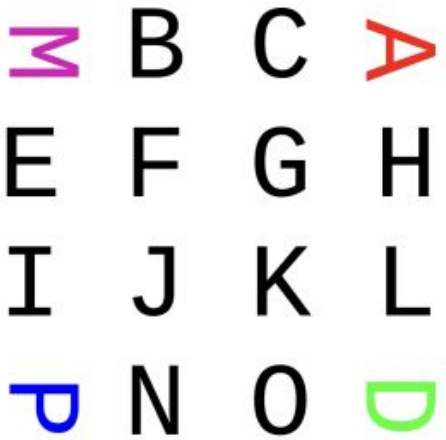
First step: from bit to block



(a) Original image



(b) Image after translating blocks



(c) Image after rotating blocks

Rotate the blocks 4 in a group -> rotate inside each block

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Some notice in advance:

- No parallelism in project 1
- No built-in intrinsics for beta submission
  - if you don't know what built-in intrinsics are, we'll cover that in upcoming recitations
- Don't jump too fast, implement one step at a time
- Check your performance by `./rotate -t tiers`
  - I'd recommend using telerun as the performance is more consistent on telerun than testing locally
- Edit your `Makefile`:
  - `CC := clang-spe`
  - `ARCH := x86-64-v3`

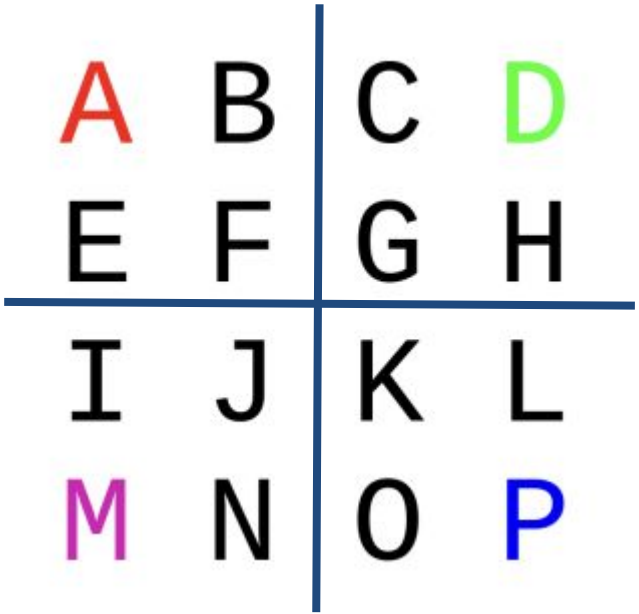
© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

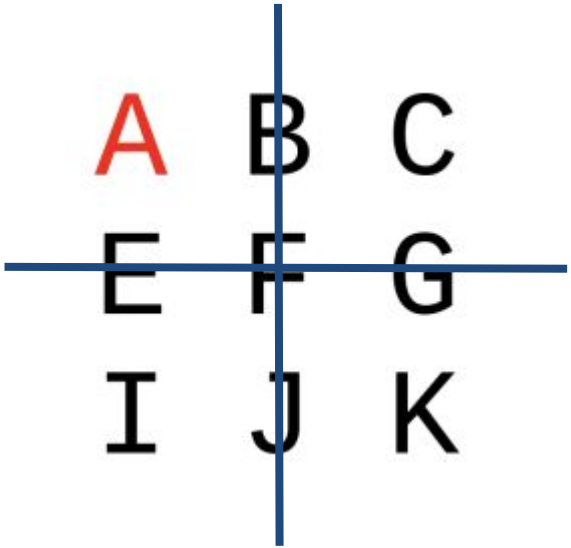
CODING TIME!



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Edge cases





© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Endianess

- Little endian: bytes of words are stored in memory with least-significant bytes first
-
- the actual word: 1A 2B 3C 4D 5E 6F 7A 8B
- store in memory: 8B 7A 6F 5E 4D 3C 2B 1A
-
- If you encounter any bugs, think about the endianess

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

CODING TIME!



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

6 recitations for project 1

1. 4-way rotation
2. bit hacks, row-column-row algorithm for column rotation, naive & divide-and-conquer using bit masks (find the excel table)
3. vectorization, built-in intrinsics (after beta submission)
4. the queen's problem using bit hacks (homework 4)
5. tiling
6. prefetching
7. performance gap table: technique, tier, engineering time estimation

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers